



SPORTS PLATFORM

Project Documentation

Version 4.0

June 2025

Confidential — For Internal Use Only

1. Project Overview

1.1 Project Description

The Sports Platform is a cross-platform application enabling users to organize, discover, and participate in local sports matches. It supports Football (Soccer), Basketball, Cricket, Tennis, Volleyball, and Badminton. Users can create or join matches, chat in real time, follow other players, view player profiles, and discover nearby games through Google Maps integration.

Field	Details
Project Name	Sports Platform
Supported Sports	Football (Soccer), Basketball, Cricket, Tennis, Volleyball, Badminton
Platforms	iOS, Android, Web (Admin Dashboard)
Backend	Python — FastAPI (async REST + WebSocket)
Database	PostgreSQL
Real-Time	WebSockets (native FastAPI)
Maps	Google Maps Platform API (Geocoding + Places)
Authentication	Email/Password + Google OAuth 2.0
Monetization	Free — No payment system

1.2 Goals & Objectives

- Provide a unified platform for organizing local sports matches across six sports.
- Enable real-time communication between players in match-specific chat rooms.
- Allow discovery of nearby matches using location-based search with advanced filters.
- Deliver a performant, scalable async API supporting both mobile clients and the web admin panel.
- Empower admins to manage users and match content from a dedicated web dashboard.
- Enable a social layer where players can follow each other and build a reputation through ratings and reviews.

1.3 Out of Scope

- Payment processing or in-app purchases.
- Video streaming or live match broadcasting.
- Sports types beyond the six defined above.
- Win/loss tracking or win rate statistics.

- Third-party analytics or advertising integrations.

2. Roles & Permissions

The platform uses a simple three-role system. Two roles are global (Regular User and Admin) and one is match-scoped (Host). There is no separate registration flow for the Host role — it is automatically assigned when a user creates a match.

2.1 Role Overview

Role	Scope	How Assigned
Regular User	Global	Auto-assigned on signup (email or Google)
Host	Match-scoped only	Auto-assigned when the user creates a match
Admin	Global	Manually created via backend — NOT via signup flow

Important: A user can be a Host in one match and a regular participant in another at the same time. The Host role only applies within the context of the match they created.

2.2 Regular User — Permissions

- Register and log in via email/password or Google OAuth.
- Create a match (becomes Host for that match automatically).
- Join and leave matches created by others.
- Discover nearby matches using filters.
- Participate in match chat rooms.
- Follow and unfollow other users.
- View other players' public profiles, ratings, and reviews.
- Leave star ratings and written reviews on other players' profiles.
- Manage their own profile (name, bio, avatar, sport skill levels).

2.3 Host — Permissions (Match-Scoped)

All Regular User permissions apply, plus the following within their own match:

- Start the match at any time using the Start Game / Ready button — regardless of whether slots are full.
- Remove any player from the match (reopens the slot).
- Edit match details (title, description, date, time, etc.).
- Delete the entire match from the Edit Match screen.
- Invite players to the match.

- The Host is automatically a participant — shown in the player list from the moment the match is created.

2.4 Admin — Permissions (Global)

Admin accounts are created manually via the backend or a secure admin creation script. Admins access the platform exclusively through the web admin dashboard — not the mobile app.

- View, search, block, unblock, and permanently remove any user.
- View and remove any match.
- View platform-wide statistics (total users, active matches, matches per sport, recent signups).
- Admin accounts are completely separate from player accounts — an admin cannot simultaneously act as a regular player.
- Admin registration is not available through the public signup flow.

2.5 Permission Matrix

Action	Regular User	Host	Admin
Register / Login	Yes	Yes	Separate flow
Create a match	Yes	Yes	No
Join a match	Yes	Auto-joined	No
Leave a match	Yes	No (owns match)	No
Start a match	No	Yes	No
Remove a player from match	No	Yes (own match)	No
Edit match details	No	Yes (own match)	No
Delete a match	No	Yes (own match)	Yes (any match)
Invite players	No	Yes	No
Follow / Unfollow users	Yes	Yes	No
Rate & review players	Yes	Yes	No
View public profiles	Yes	Yes	Yes
Block / Remove users	No	No	Yes
Access admin dashboard	No	No	Yes
View platform statistics	No	No	Yes

3. System Architecture

3.1 High-Level Architecture

The platform follows a client-server architecture with a clear separation between the mobile clients, web admin dashboard, and the backend API layer. All communication flows through the FastAPI backend which exposes both REST endpoints and WebSocket connections.

Layer	Technology	Responsibility
Mobile App	iOS / Android (React Native or Flutter)	User-facing mobile experience
Web Dashboard	React.js (Browser)	Admin panel for managing users & matches
API Gateway	FastAPI (Python 3.11+)	Routes requests, auth, validation
Business Logic	FastAPI Services / Routers	Core domain logic, async handlers
Real-Time Layer	FastAPI WebSockets	Chat messages, live notifications
Data Layer	PostgreSQL + SQLAlchemy (async)	Persistent storage for all entities
Cache Layer	Redis (Phase 2)	Session tokens, pub/sub for WebSockets
File Storage	AWS S3 / Cloudinary	Profile pictures
Maps & Geocoding	Google Maps Platform API	Address geocoding, match location display
Auth Provider	Google OAuth 2.0 + JWT	Authentication & token management

3.2 Architecture Notes

- Mobile App communicates with FastAPI via HTTPS REST and persistent WebSocket connections.
- Web Admin Dashboard communicates with the same FastAPI backend using admin-scoped JWT tokens.
- All requests pass through middleware (CORS, Auth, Rate Limiting) before reaching routers.
- Google Maps Geocoding API is called server-side when a match location address is submitted.

- File uploads use pre-signed URLs — clients upload directly to cloud storage without going through the API server.

3.3 Deployment Architecture

Component	Deployment Target	Notes
FastAPI App	Docker on cloud (AWS/GCP)	Uvicorn ASGI server, multiple workers
PostgreSQL	Managed DB (RDS / Cloud SQL)	asyncpg connection pool
Redis	Managed Redis (ElastiCache)	WebSocket pub/sub & caching (Phase 2)
Static Files	AWS S3 + CloudFront CDN	Profile pictures
Admin Dashboard	Vercel / Netlify	Static React build
Mobile Apps	App Store & Google Play	Published mobile builds

4. Project Flow

4.1 User Registration & Authentication Flow

Covers: Email signup, Google OAuth, JWT token issuance

4.1.1 Email Registration

- User submits email, password, and full name.
- Backend validates input (unique email, password strength).
- Password is hashed using bcrypt.
- User record is created with status PENDING_VERIFICATION.
- Verification email is dispatched via background task (async).
- User verifies email — status updated to ACTIVE.
- JWT access token + refresh token issued on successful login.

4.1.2 Google OAuth Login

- Client receives Google ID token via Google Sign-In SDK.
- Client sends ID token to POST /auth/google.
- Backend verifies token with Google's public keys.
- If user exists: issue JWT tokens and return profile.
- If new user: auto-create account, issue tokens, prompt profile completion.

4.2 Create Match Flow

Covers: Form input, validation, host auto-join, match creation

- User opens Create Match screen.
- User fills in Match Title (text input) and Description (text input).
- User selects all required fields via pickers/dropdowns: Sport Type, Date, Time, Match Duration, Location (address search), Maximum Players, Skill Level.
- The Create Match button remains disabled until all required fields are completed.
- User submits — POST /matches is called with all field values.
- Backend geocodes the entered facility address via Google Maps Geocoding API to obtain lat/lng coordinates.
- Match record is created in PostgreSQL with status OPEN.
- The creator is automatically added as a participant (Host) in the MatchPlayer table.
- The match immediately appears in the host's My Matches list.
- The host is shown as a participant in the match player list.

4.3 Match Lifecycle & Host Control Flow

Covers: Join → Full → Host decision → Start or Remove → Ongoing → Complete

Stage	Status	Description	Actor
Match Created	OPEN	Host creates match, is auto-joined as participant. Slots available.	Host (auto)
Players Join	OPEN	Players discover and join the match freely.	Players
Slots Full	FULL	Max capacity reached. System sets status to FULL automatically. Game does NOT start automatically.	System
Host Reviews	FULL	Host reviews the player list. Can remove a wrongly joined player if needed.	Host
Slot Reopened	OPEN	If host removes a player, one slot reopens and status returns to OPEN.	Host
Host Starts Game	ONGOING	Host presses Start Game / Ready at any time (slots do not need to be full). Game begins.	Host
Match Live	ONGOING	Match is live. No new players can join. Real-time chat is active.	Players
Match Completed	COMPLETE D	Host marks match as done. Post-match ratings and reviews are enabled.	Host
Match Cancelled	CANCELLE D	Host cancels the match before it starts.	Host

Key Rule: The game **ONLY** starts when the host explicitly presses Start Game / Ready. A full match (e.g. 10/10 slots) does **NOT** trigger an automatic start. The host always has final control over who participates before the game begins.

4.4 Discovery & Filter Flow

Covers: Location permission, filter defaults, search, results

- User opens the Discover screen — location permission is requested if not already granted.
- Default filter values are applied automatically on load:
 - Sport: auto-set to the sport the user is currently browsing.
 - Distance: 20 km radius from user's current location.
 - Skill Level: Any (no restriction).

- Date & Time: All upcoming matches (no date filter applied).
- User can open the filter panel to adjust any of the four filters.
- App calls GET /matches/nearby with query params: lat, lng, radius, sport, skill_level, date_from, date_to.
- Backend performs a geospatial radius query on PostgreSQL, filtered by selected criteria.
- Results are returned sorted by distance, limited to status OPEN or FULL.
- User taps a match card to view full details, player list, and join button.

4.5 Real-Time Chat Flow

- User opens match chat — client connects to WebSocket: WS /ws/matches/{match_id}/chat.
- Server authenticates the WebSocket handshake using JWT passed as a query parameter.
- Server validates that the user is an active participant in the match.
- Client sends JSON message payload over the WebSocket connection.
- Server broadcasts message to all connected clients in that match room.
- Message is persisted to PostgreSQL asynchronously via background task.
- On disconnect: connection is removed from the WebSocket manager.

4.6 Follow & Profile Flow

- User searches for or views another user's public profile.
- Profile displays: name, location, bio, avatar, followers count, following count, average star rating, total games played, sport skill levels (per sport), and reviews.
- User taps Follow — POST /users/{user_id}/follow is called.
- Follower/following counts are updated for both users.
- User taps Message — opens a direct chat or redirects to match chat.
- After a completed match, participants can leave a star rating (1–5) and written review on each other's profiles.

4.7 Notification Flow

- Notifications are triggered by system events: match joined, player invited, match status changed, new chat message, new follower.
- Backend creates a Notification record and pushes to the user's active WebSocket channel.
- If the user is offline, the notification is stored and delivered on next connection.
- Push notifications (FCM for Android, APNs for iOS) are sent for offline users via background tasks.

5. API Structure & Behavior

All REST endpoints are asynchronous, versioned under /api/v1, and return JSON. Authentication uses Bearer JWT tokens. WebSocket endpoints are under /ws/. All timestamps are UTC ISO 8601.

5.1 Base URL & Versioning

Field	Value
REST Base URL	https://api.sportsplatform.com/api/v1
WebSocket Base	wss://api.sportsplatform.com/ws
Content Type	application/json
Auth Header	Authorization: Bearer <JWT_ACCESS_TOKEN>

5.2 Authentication Endpoints

Method	Endpoint	Description	Auth
POST	/auth/register	Register with email & password	No
POST	/auth/login	Login, receive JWT tokens	No
POST	/auth/google	Google OAuth login/signup	No
POST	/auth/refresh	Refresh access token	Refresh Token
POST	/auth/logout	Revoke refresh token	Yes
POST	/auth/verify-email	Verify email with token	No
POST	/auth/forgot-password	Request password reset	No
POST	/auth/reset-password	Set new password	No

5.3 User & Profile Endpoints

Method	Endpoint	Description	Auth
GET	/users/me	Get current user profile	Yes
PUT	/users/me	Update profile (name,	Yes

		bio, sport skill levels)	
POST	/users/me/avatar	Upload profile picture	Yes
GET	/users/{user_id}	View another user's public profile	Yes
GET	/users/{user_id}/reviews	Get ratings & reviews for a user	Yes
POST	/users/{user_id}/reviews	Post a star rating & written review	Yes
GET	/users/{user_id}/stats	Get games played count	Yes
POST	/users/{user_id}/follow	Follow a user	Yes
DELETE	/users/{user_id}/follow	Unfollow a user	Yes
GET	/users/{user_id}/followers	Get followers list	Yes
GET	/users/{user_id}/following	Get following list	Yes

5.4 Match Endpoints

Method	Endpoint	Description	Auth
POST	/matches	Create a new match (host auto-joined)	Yes
GET	/matches/{match_id}	Get match details	Yes
PUT	/matches/{match_id}	Edit match details (host only)	Yes (host)
DELETE	/matches/{match_id}	Delete the entire match (host only)	Yes (host)
POST	/matches/{match_id}/join	Join a match	Yes
DELETE	/matches/{match_id}/leave	Leave a match	Yes
GET	/matches/{match_id}/players	List participants in a match	Yes
DELETE	/matches/{match_id}/players/{user_id}	Remove a player (host only)	Yes (host)
PATCH	/matches/{match_id}/status	Update match status — Start Game sets ONGOING	Yes (host)
POST	/matches/{match_id}/	Invite a player to the	Yes (host)

	invite	match	
--	--------	-------	--

5.5 Discovery Endpoints

Method	Endpoint	Description	Auth
GET	/matches/nearby	Nearby matches with filters (see filter params below)	Yes
GET	/matches	List matches with filters (sport, status, date)	Yes

5.5.1 Discovery Filter Query Parameters

Parameter	Type	Default	Description
lat	float (required)	—	User's current latitude
lng	float (required)	—	User's current longitude
radius_km	integer	20	Search radius in km (default 20 km)
sport	string	Current sport context	Filter by sport type
skill_level	string	any	Beginner / Intermediate / Advanced / any
date_from	ISO 8601 date	Now (upcoming)	Start of date range filter
date_to	ISO 8601 date	null (no limit)	End of date range filter
page	integer	1	Pagination — page number
limit	integer	20	Pagination — results per page

5.6 Chat & WebSocket Endpoints

Type	Endpoint	Description	Auth
WS	/ws/matches/{match_id}/chat	Real-time match chat room	JWT query param
GET	/matches/{match_id}/messages	Fetch chat history (paginated)	Yes
WS	/ws/notifications	Real-time notification stream	JWT query param

5.6.1 WebSocket Message Schema

Outbound (client → server):

```
{ "type": "chat_message", "content": "See you at 5pm!" }
```

Inbound (server → client):

```
{ "type": "chat_message", "sender_id": "uuid", "sender_name": "Ali Khan",  
  "content": "See you at 5pm!", "sent_at": "2025-06-01T14:00:00Z" }
```

5.7 Notification Endpoints

Method	Endpoint	Description	Auth
GET	/notifications	Get user notifications (paginated)	Yes
PATCH	/notifications/{id}/read	Mark notification as read	Yes
PATCH	/notifications/read-all	Mark all as read	Yes

5.8 Admin Endpoints

Method	Endpoint	Description	Auth
GET	/admin/users	List all users with filters	Admin JWT
GET	/admin/users/{user_id}	Get full user details	Admin JWT
DELETE	/admin/users/{user_id}	Permanently remove a user	Admin JWT
PATCH	/admin/users/{user_id}/block	Block / unblock a user	Admin JWT
GET	/admin/matches	List all matches	Admin JWT
DELETE	/admin/matches/{match_id}	Remove a match	Admin JWT
GET	/admin/dashboard/stats	Platform statistics overview	Admin JWT

6. Data Models

All models are stored in PostgreSQL. UUIDs are used as primary keys. Timestamps are stored in UTC. SQLAlchemy async ORM is used for all database interactions.

6.1 User Model

Field	Type	Description
id	UUID (PK)	Unique user identifier
email	VARCHAR(255) UNIQUE	User email address
hashed_password	VARCHAR NULLABLE	Bcrypt hashed password (null for Google users)
google_id	VARCHAR UNIQUE NULLABLE	Google OAuth subject ID
full_name	VARCHAR(100)	Display name
bio	TEXT NULLABLE	Short user biography
location	VARCHAR(100) NULLABLE	City / location label
avatar_url	VARCHAR NULLABLE	Profile picture URL
avg_rating	FLOAT DEFAULT 0.0	Computed average star rating from reviews
total_games_played	INTEGER DEFAULT 0	Total completed matches participated in
status	ENUM (Active / Blocked)	Account status
is_admin	BOOLEAN DEFAULT false	Admin privilege flag
created_at	TIMESTAMPTZ	Account creation timestamp

6.2 UserSport Model (Skill Per Sport)

Each user can have a different skill level per sport. This replaces a single global skill level.

Field	Type	Description
id	UUID (PK)	Record identifier
user_id	UUID (FK → users)	The user this record belongs to
sport	ENUM (Football/Basketball/Cricket/Tennis/Volleyball/Badminton)	Sport type

skill_level	ENUM (Beginner / Intermediate / Advanced)	User's skill level for this sport
-------------	---	-----------------------------------

6.3 Follow Model

Field	Type	Description
follower_id	UUID (FK → users)	The user who is following
following_id	UUID (FK → users)	The user being followed
created_at	TIMESTAMPTZ	When the follow occurred

6.4 Match Model

Field	Type	Description
id	UUID (PK)	Unique match identifier
host_id	UUID (FK → users)	Match host (creator)
sport	ENUM (Football/Basketball/Cricket/Tennis/Volleyball/Badminton)	Sport type
title	VARCHAR(150)	Match title
description	TEXT NULLABLE	Additional match details
facility_address	VARCHAR(255)	User-entered facility address (raw text)
location_name	VARCHAR(255)	Resolved human-readable location label
latitude	FLOAT	Geocoded GPS latitude
longitude	FLOAT	Geocoded GPS longitude
scheduled_at	TIMESTAMPTZ	Match date and time
duration_minutes	INTEGER	Match duration in minutes
max_players	INTEGER	Maximum player capacity (not a start requirement)
skill_level	ENUM (Beginner / Intermediate / Advanced)	Preferred skill level for this match
status	ENUM (Open / Full / Ongoing / Completed / Cancelled)	Match status
created_at	TIMESTAMPTZ	Record creation timestamp

6.5 Supporting Models

MatchPlayer (Junction Table)

Field	Type	Description
match_id	UUID (FK → matches)	Match reference
user_id	UUID (FK → users)	Player reference
role	ENUM (Host / Player)	Role in this match
joined_at	TIMESTAMPTZ	When the player joined
status	ENUM (Active / Removed / Left)	Participation status

Review Model

Field	Type	Description
id	UUID (PK)	Review identifier
reviewer_id	UUID (FK → users)	User leaving the review
reviewee_id	UUID (FK → users)	User receiving the review
match_id	UUID (FK → matches)	The match context for this review
rating	INTEGER (1–5)	Star rating
comment	TEXT NULLABLE	Written review text
created_at	TIMESTAMPTZ	Review timestamp

Message Model

Field	Type	Description
id	UUID (PK)	Message identifier
match_id	UUID (FK → matches)	Match chat room
sender_id	UUID (FK → users)	Message sender
content	TEXT	Message content
sent_at	TIMESTAMPTZ	Send timestamp

Notification Model

Field	Type	Description
id	UUID (PK)	Notification identifier

user_id	UUID (FK → users)	Recipient
type	VARCHAR(50)	Event type (match_joined, invited, new_follower, etc.)
payload	JSONB	Dynamic data (match_id, sender_name, etc.)
is_read	BOOLEAN DEFAULT false	Read status
created_at	TIMESTAMPTZ	Creation timestamp

7. Feature Modules

7.1 User Accounts Module

- Email/password registration with bcrypt hashing and email verification.
- Google OAuth 2.0 login — ID token verified server-side.
- Profile: full name, avatar, bio, city/location, and per-sport skill levels.
- Skill level is defined per sport — a user can be Intermediate in Basketball and Beginner in Tennis.

7.2 Match Module

- A match contains: sport, title, description, facility address (geocoded), date, time, duration, max players, and skill level.
- Supported sports: Football, Basketball, Cricket, Tennis, Volleyball, Badminton.
- Max players is a capacity ceiling — NOT a start requirement.
- When max capacity is reached, status is automatically set to FULL by the system — but the game does NOT start.
- The host retains full control: can remove a player (reopens slot, status → OPEN) or start the game at any time.
- Host Controls Panel (same section): Start Game, Remove Player, Edit Match, Delete Match.
- The creator is automatically added as a participant (Host role) at the moment of match creation.
- The match immediately appears in the host's My Matches list upon creation.
- Delete Match is accessible from the Edit Match screen (host only).

7.3 Create Match Form

Field	Input Type	Notes
Match Title	Text Input	Required — free text
Description	Text Input (multiline)	Required — free text
Sport Type	Selectable cards / Bottom Sheet	Required — no manual input
Date	Date Picker	Required — no manual input
Time	Time Picker	Required — no manual input
Match Duration	Picker / Stepper	Required — no manual input
Location	Address Search	Required — user types address, system

	(Google Maps)	geocodes
Maximum Players	Stepper / Dropdown	Required — no manual input
Skill Level	Bottom Sheet / Dropdown	Required — Beginner / Intermediate / Advanced

- The Create Match button is disabled until all required fields are filled.
- No image upload — text and selections only.

7.4 Discovery & Filter Module

- Location-based search returns OPEN and FULL matches within the selected radius.
- Uses geospatial query on PostgreSQL (Haversine / PostGIS) with indexed lat/lng columns.

Filter Options & Defaults

Filter	Options	Default
Sport Type	Football, Basketball, Cricket, Tennis, Volleyball, Badminton	Current sport being browsed
Distance	Slider or input in km	20 km
Skill Level	Beginner / Intermediate / Advanced / Any	Any
Date & Time	Date picker / range selector	All upcoming matches (no filter)

7.5 Follow & Social Module

- Users can follow and unfollow other players.
- Profile displays follower count and following count.
- A Message button on the profile opens a direct chat or redirects to the match chat.
- Following a user does not require approval — open follow system.
- New follower events trigger a notification for the followed user.

7.6 Ratings & Reviews Module

- After a match is marked COMPLETED, participants can rate and review each other.
- Reviews consist of a star rating (1–5 stars) and an optional written comment.
- A user can only review another user once per match.
- Reviews are visible on the reviewed user's public profile.
- The profile displays: average star rating, total number of reviews, total games played.

- Win rate percentage is NOT tracked or displayed.

7.7 Maps & Location Module

- During match creation, the user types a facility address (e.g. Nørrebrogade 285, 2200 Nørrebro).
- The backend calls Google Maps Geocoding API to convert the address to lat/lng coordinates.
- Both the raw address and resolved coordinates are stored on the Match record.
- Mobile app renders a Google Maps view with pins for nearby matches on the Discover screen.
- Match detail screen shows the facility address and a map pin for the location.

7.8 Real-Time Chat Module

- Each match has a dedicated chat room accessible to all participants.
- Chat is powered by FastAPI native WebSockets.
- Messages are persisted in PostgreSQL and paginated (50 per page).
- Chat history is available to participants via REST (GET /matches/{match_id}/messages).

7.9 Admin Panel Module

- Separate React.js web dashboard, accessible only to admin accounts.
- User management: view, search, block, unblock, permanently delete users.
- Match management: view all matches, filter by sport/status, remove matches.
- Dashboard statistics: total users, active matches, matches per sport, recent signups.

8. Async API Design & Behavior

8.1 Async Architecture Principles

The backend is built entirely on FastAPI with async/await patterns. All I/O operations — database queries, external API calls, and WebSocket communication — are non-blocking.

Principle	Implementation
Async DB Queries	SQLAlchemy 2.x with asyncpg — all queries use await
Async HTTP	httpx.AsyncClient for Google Maps API and Google OAuth
Background Tasks	FastAPI BackgroundTasks for emails, push notifications, DB writes
WebSocket Manager	In-memory connection manager with asyncio locks
Dependency Injection	FastAPI Depends() for DB sessions, auth, role checks
Connection Pooling	asyncpg pool via SQLAlchemy async engine (min 5, max 20)

8.2 WebSocket Connection Manager

- `connect(match_id, websocket)`: adds connection to the match room.
- `disconnect(match_id, websocket)`: removes connection from the room.
- `broadcast(match_id, message)`: sends message to all connections in the room asynchronously.
- `send_personal(user_id, message)`: sends to a specific user's notification WebSocket.

8.3 HTTP Response Codes

Code	Meaning	Common Trigger
200 OK	Success	GET, successful update
201 Created	Resource created	POST match, register
400 Bad Request	Validation error	Invalid input, missing fields
401 Unauthorized	Missing/invalid JWT	No or expired token
403 Forbidden	Insufficient permissions	Non-host modifying match

404 Not Found	Resource missing	Match or user not found
409 Conflict	Duplicate resource	Email already registered
422 Unprocessable	Pydantic validation fail	Wrong data types
429 Too Many Requests	Rate limit exceeded	Abuse prevention on auth routes
500 Internal Error	Unhandled exception	DB failure, unexpected error

8.4 Authentication & Authorization

- Access tokens expire after 15 minutes. Refresh tokens expire after 30 days.
- JWT validated on each request via FastAPI dependency (`get_current_user`).
- Admin endpoints use a separate dependency (`get_current_admin`) checking `is_admin=True`.
- Host-only endpoints verify the requesting user is the match `host_id`.
- WebSocket handshakes authenticate via JWT passed as a query parameter: `?token=<access_token>`.

9. Non-Functional Requirements

9.1 Performance

- API response time under 300ms for standard REST endpoints at normal load.
- WebSocket message delivery under 100ms within the same region.
- Discovery endpoint must handle geospatial queries efficiently over large match datasets.

9.2 Scalability

- Stateless REST API — horizontal scaling via multiple Uvicorn workers or container instances.
- WebSocket state per-instance; Redis pub/sub added in Phase 2 for multi-instance scaling.
- Database connection pooling with asyncpg.

9.3 Security

- All traffic over HTTPS / WSS (TLS 1.2+).
- Passwords hashed with bcrypt (cost factor 12).
- JWT tokens signed with RS256.
- Input validated via Pydantic schemas on all endpoints.
- Rate limiting on auth endpoints (5 requests/minute per IP).
- Admin endpoints accessible only to admin-scoped JWT tokens.

9.4 Availability

- Target uptime: 99.5% (excluding planned maintenance).
- Graceful WebSocket reconnection with exponential backoff on the client.
- Managed cloud database with automated backups and failover.

9.5 Maintainability

- Domain-driven folder structure: /routers, /services, /models, /schemas, /dependencies.
- Auto-generated OpenAPI docs at /docs (Swagger UI).
- Environment-based configuration using pydantic-settings.

10. Development Phases

Phase	Name	Key Deliverables
Phase 1	Foundation	Project setup, DB schema, Auth (email + Google), User profiles, Per-sport skill levels
Phase 2	Core Match Features	Create/join/leave match, host controls (start, remove player), match status management
Phase 3	Discovery & Maps	Location-based search, discovery filters (sport, distance, skill, date), Google Maps geocoding
Phase 4	Real-Time Features	WebSocket chat, notifications (in-app + push via FCM/APNs)
Phase 5	Social Features	Follow/unfollow system, player invitations, ratings & reviews
Phase 6	Admin Panel	Admin web dashboard: user & match management, platform statistics
Phase 7	QA & Hardening	Testing, performance tuning, security audit, bug fixes
Phase 8	Deployment	Production setup, CI/CD pipeline, app store submission

10.1 Technology Summary

Category	Technology / Tool
Language	Python 3.11+
Framework	FastAPI
ASGI Server	Uvicorn
ORM	SQLAlchemy 2.x (async mode)
DB Driver	asyncpg
Database	PostgreSQL 15+
Auth	python-jose (JWT), passlib (bcrypt), httpx (Google OAuth)
WebSockets	FastAPI native WebSocket support
Background Tasks	FastAPI BackgroundTasks → Celery + Redis

	(Phase 2+)
Push Notifications	Firebase Cloud Messaging (FCM) + Apple Push Notifications (APNs)
Geocoding	Google Maps Geocoding API (server-side)
Maps Display	Google Maps Platform (mobile & web)
File Storage	AWS S3 / Cloudinary
Testing	pytest + pytest-asyncio + httpx AsyncClient
API Docs	FastAPI auto-generated OpenAPI / Swagger UI
Mobile	React Native or Flutter (iOS + Android)
Admin Web	React.js

11. Glossary

Term	Definition
JWT	JSON Web Token — compact, self-contained token for authentication
OAuth 2.0	Authorization framework used for Google login
WebSocket	Full-duplex communication protocol over a persistent TCP connection
ASGI	Asynchronous Server Gateway Interface — async Python server standard
Host	A regular user who created a specific match — match-scoped role only
Match-Scoped	A role or permission that only applies within a specific match context
Geocoding	Converting a text address into geographic coordinates (lat/lng) via Google Maps API
asyncpg	High-performance async PostgreSQL driver for Python
Match Room	A logical group of WebSocket connections tied to a single match for chat
FCM	Firebase Cloud Messaging — Google's push notification service for Android
APNs	Apple Push Notification service for iOS devices
PostGIS	PostgreSQL extension for geospatial queries (nearby match search)
Pydantic	Python data validation library used for request/response schemas in FastAPI
Uvicorn	ASGI server used to run the FastAPI application
Per-Sport Skill	A skill level defined individually per sport, not as a single global value

End of Document — Sports Platform v4.0

12. Backend Architecture & Development Standards

The project follows a clean, scalable FastAPI architecture designed for maintainability, performance, and long-term clarity. Every component has a single, well-defined responsibility. The goal is a production-ready backend that is easy to extend, debug, and scale.

12.1 Folder Structure

The project is organized into clearly separated directories. Each directory owns a specific layer of the application and must not mix concerns with other layers.

Directory / File	Responsibility
app/	Root application package
app/main.py	FastAPI app initialization, middleware registration, router inclusion
app/config.py	Environment-based configuration using pydantic-settings
app/dependencies.py	Shared FastAPI dependencies: DB session, current user, admin check
app/routes/	API route definitions — thin layer, delegates all logic to services
app/services/	Business logic — all domain operations live here
app/models/	SQLAlchemy ORM models — maps directly to PostgreSQL tables
app/schemas/	Pydantic request/response schemas — controls what data enters and exits the API
app/utlis/	Shared utilities: password hashing, JWT helpers, email sender, geocoding client
app/websockets/	WebSocket connection manager and event handlers
app/background/	Background task definitions: push notifications, email dispatch, DB writes
tests/	Async test suite using pytest + pytest-asyncio + httpx AsyncClient

12.2 Layer Responsibilities

Routes Layer (app/routes/)

- Defines HTTP method, path, and expected request/response schemas.
- Calls the appropriate service function — contains no business logic itself.
- Injects dependencies (current user, DB session) via FastAPI Depends().
- Each feature module has its own router file: auth.py, matches.py, users.py, admin.py, etc.

Services Layer (app/services/)

- Contains all business logic and domain rules.
- Interacts with the database via SQLAlchemy async ORM.

- Calls external APIs (Google Maps Geocoding, Google OAuth) using `httpx.AsyncClient`.
- Raises structured HTTP exceptions that propagate cleanly to the routes layer.
- Triggers background tasks for non-blocking operations.

Models Layer (`app/models/`)

- Defines SQLAlchemy ORM classes that map to PostgreSQL tables.
- Uses UUID primary keys and TIMESTAMPTZ for all timestamps.
- Relationships defined with SQLAlchemy `relationship()` and FK constraints.
- Indexes applied to frequently queried columns: `lat/lng`, `sport`, `status`, `scheduled_at`.

Schemas Layer (`app/schemas/`)

- Pydantic v2 models used for all request bodies and API responses.
- Separate schemas for Create, Update, and Response to avoid over-exposing data.
- Response schemas return only the exact fields needed by the client — no extra data.
- Passwords, internal flags, and sensitive fields are never included in response schemas.

Utils Layer (`app/utils/`)

- `password.py` — bcrypt hashing and verification using `passlib`.
- `jwt.py` — JWT token creation and validation using `python-jose`.
- `email.py` — async email dispatch helper.
- `geocoding.py` — async wrapper around Google Maps Geocoding API.
- `pagination.py` — shared pagination utility for list endpoints.

12.3 Error Handling Standards

All errors must be caught and returned as structured JSON responses. The application must never crash due to an unhandled exception. FastAPI's exception handler system is used globally.

Error Type	Handling Approach
Validation errors (Pydantic)	FastAPI returns 422 automatically with field-level detail
Authentication errors	Raise <code>HTTPException(401)</code> in <code>get_current_user</code> dependency
Authorization errors	Raise <code>HTTPException(403)</code> in service layer after ownership check
Resource not found	Raise <code>HTTPException(404)</code> in service layer when DB query returns None

Conflict (duplicate data)	Raise HTTPException(409) on unique constraint violations
External API failures	Caught in utils layer, raise HTTPException(502) with safe message
Unhandled exceptions	Global exception handler returns 500 with a generic safe message
WebSocket errors	Caught per-connection, close with appropriate code, log server-side

- Error responses always follow a consistent structure: { detail: string } or { detail: [{ field, message }] }.
- Internal error details (stack traces, DB errors) are never exposed to the client.
- All exceptions are logged server-side with sufficient context for debugging.

12.4 Async Standards

- Every route handler, service function, and DB query must use `async def` and `await`.
- All database operations use SQLAlchemy 2.x async session (`AsyncSession`).
- External HTTP calls (Google Maps, Google OAuth) use `httpx.AsyncClient` with connection pooling.
- No blocking I/O (no `requests` library, no `time.sleep`) is permitted anywhere in the codebase.
- WebSocket handlers are fully async — broadcast and send operations use `await`.

12.5 Rate Limiting

Rate limiting is applied at the route level using a FastAPI-compatible middleware (e.g. `slowapi`). Limits are configured per endpoint category to balance protection and usability.

Endpoint Category	Rate Limit	Reason
Auth endpoints (register, login, reset)	5 requests / minute / IP	Prevent brute force and spam
Match creation	10 requests / minute / user	Prevent spam match creation
Discovery / search	30 requests / minute / user	Allow frequent browsing
Review submission	10 requests / minute / user	Prevent review spam
General API endpoints	60 requests / minute / user	General abuse protection
WebSocket connections	5 connections / user	Prevent connection flooding

- Rate limit exceeded responses return HTTP 429 with a `Retry-After` header.

- IP-based limiting used for unauthenticated endpoints; user-based for authenticated ones.

12.6 Background Tasks

Background tasks handle all non-blocking, non-critical operations that do not need to complete before the API returns a response. FastAPI BackgroundTasks is used in Phase 1, with Celery + Redis introduced in Phase 2 for heavier workloads.

Task	Trigger	Handler
Send verification email	User registers via email	Background task after user creation
Send password reset email	User requests password reset	Background task after token generation
Geocode match address	Match is created	Background task — updates lat/lng after creation
Send push notification (FCM/APNs)	Match joined, invited, status changed	Background task after DB event
Persist chat message to DB	WebSocket message received	Background task after broadcast
Update games played count	Match marked COMPLETED	Background task updates all participant stats
Update average rating	New review submitted	Background task recomputes avg_rating on user

- Background tasks are always triggered after the primary operation succeeds — they never block the response.
- Failed background tasks are logged with full context. Critical tasks (emails, notifications) are retried via Celery in Phase 2.

12.7 Response Design

APIs return only the exact fields required by the client. Response schemas are purpose-built for each endpoint — never returning raw DB model objects directly.

- Separate Pydantic response schemas for each use case (e.g. MatchSummaryResponse for lists, MatchDetailResponse for detail view).
- List endpoints return paginated responses with the structure: { items: [...], total: int, page: int, limit: int }.
- Sensitive fields (hashed_password, is_admin, internal flags) are excluded from all response schemas.
- Null / optional fields are omitted from responses where possible (model_config with exclude_none=True).

- Nested objects (e.g. host profile inside match response) use lightweight summary schemas, not full models.

12.8 Database Standards

- All tables use UUID primary keys generated server-side (uuid4).
- All timestamps stored as TIMESTAMPTZ (UTC) — never naive datetimes.
- Foreign key constraints enforced at the DB level, not only in the ORM.
- Indexes applied to: lat/lng columns (geospatial queries), sport + status + scheduled_at (discovery filters), user_id on all junction tables.
- N+1 query problems avoided by using SQLAlchemy selectinload() or joinedload() for related data.
- Database migrations managed via Alembic with versioned migration scripts.
- Raw SQL used only when ORM cannot express the query efficiently (e.g. complex geospatial calculations).

13. Pagination

All list endpoints in the platform use cursor-free, page-based pagination. Every paginated endpoint accepts the same standard query parameters and returns the same standard response envelope. Pagination logic is centralized in `app/utils/pagination.py` and reused across all endpoints.

13.1 Standard Query Parameters

Parameter	Type	Default	Description
page	integer	1	Page number — starts at 1
limit	integer	20	Number of items per page — default 20, max 100

- If limit exceeds 100, the server clamps it to 100 and returns a warning in the response.
- If page is less than 1, the server returns HTTP 400 Bad Request.

13.2 Standard Pagination Response Envelope

Every paginated endpoint wraps its results in the following structure. The items array contains the actual data objects for the current page.

```
{
  "items":    [...],    // array of result objects for this page
  "total":    120,      // total number of matching records
  "page":     1,        // current page number
  "limit":    20,       // items per page
  "has_next": true,     // true if a next page exists
  "has_prev": false    // true if a previous page exists
}
```

13.3 Paginated Endpoints

The following endpoints all support standard pagination query parameters (`?page`, `?limit`) and return the standard pagination envelope.

Method	Endpoint	Default Limit	Notes
GET	<code>/matches/nearby</code>	20	Sorted by distance from user. Filters: sport, radius, skill level,

			date.
GET	/matches	20	General match listing. Filters: sport, status, date range.
GET	/matches/{match_id}/players	20	Lists active participants in a match.
GET	/matches/{match_id}/messages	20	Chat history — sorted newest first (reverse chronological).
GET	/users/{user_id}/reviews	20	Reviews on a player profile — sorted newest first.
GET	/users/{user_id}/followers	20	List of users following this profile.
GET	/users/{user_id}/following	20	List of users this profile is following.
GET	/notifications	20	User notifications — sorted newest first, unread first.
GET	/admin/users	20	Admin: full user list. Filters: status, search by name/email.
GET	/admin/matches	20	Admin: full match list. Filters: sport, status, date range.

13.4 Pagination Utility (app/utils/pagination.py)

A shared utility function handles all pagination logic centrally. Routes and services call this utility instead of implementing pagination independently.

Function / Class	Purpose
paginate(query, page, limit, session)	Executes a SQLAlchemy async query with LIMIT/OFFSET and returns the full pagination envelope
PaginationParams (Depends)	FastAPI dependency that extracts and validates page and limit from query params
PaginatedResponse[T] (Pydantic generic)	Generic Pydantic response model used as the return type for all paginated endpoints

- PaginationParams is injected via FastAPI Depends() — routes never manually extract page/limit from the request.
- PaginatedResponse[T] is a generic Pydantic model where T is the item schema (e.g. PaginatedResponse[MatchSummaryResponse]).
- The paginate() utility runs two queries: one for the data (with LIMIT/OFFSET) and one COUNT query for the total — both async.
- OFFSET-based pagination is used. For very large datasets, keyset/cursor pagination can be adopted in a future phase.

13.5 Example Request & Response

Request

```
GET /api/v1/matches/nearby?
lat=55.69&lng=12.56&radius_km=20&sport=Basketball&page=2&limit=20
```

Response

```
{
  "items": [ { "id": "uuid", "title": "Evening Basketball", "distance_km":
3.2, ... }, ... ],
  "total": 45,
  "page": 2,
  "limit": 20,
  "has_next": true,
  "has_prev": true
}
```